



TA : Amany Sherif

Book Orbit

A .NET Clean Architecture BackEnd System.

Connecting students through the
orbit of knowledge sharing.



Team Members

Back End members

Adham Eltantawi
(Lead)

Eyad Dawood
(Backend Lead)

Abdelrahman
Mohye

Front End members

Abdelrahman Yasser
(Frontend Lead)

Omar Ashraf

Abdelrahman Nofal

The Problem & Why It's Hard



Financial Waste

Students waste money on textbooks by abandoning them after a being read once



State Complexity

Peer-to-peer lending requires strict transaction state integrity — one missed transition breaks the entire chain



Zero Trust & Identity Mandate

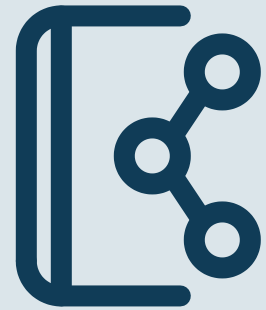
No trusted, centralized platform exists for secure student-to-student lending. Identity trust is non-negotiable — only verified university students can participate in the ecosystem.

The Solution: BookOrbit at a Glance



- **Verify Email & Admin Approve**

Secures the platform through admin-gated onboarding utilizing mandatory university email verification.



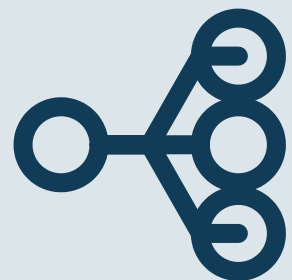
- **Shared Catalog**

Separates the shared Book catalog from physical BookCopy ownership to enforce a single source of truth.



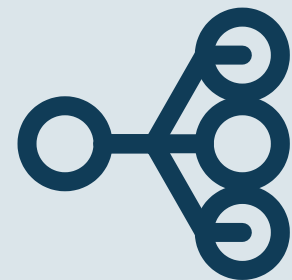
- **Browse & Request**

Allows students to list physical book copies while others browse, request, and manage borrowing lifecycles.



- **State Transitions**

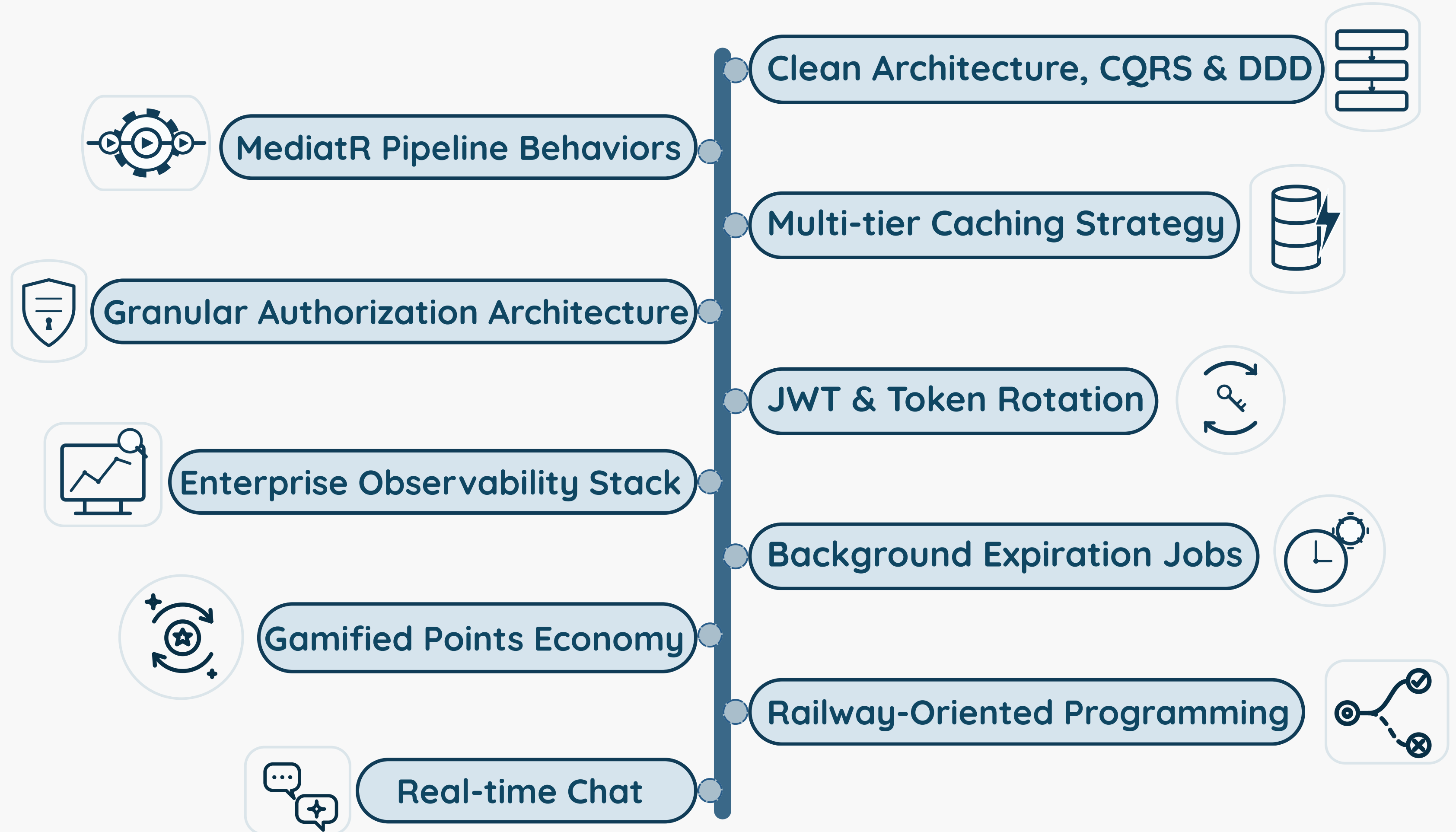
Manages the complete borrowing lifecycle through strict states: pending, accepted, rejected, or cancelled.



- **OTP Verification**

Ensures secure handoff and return using a time-limited code validated by both parties.

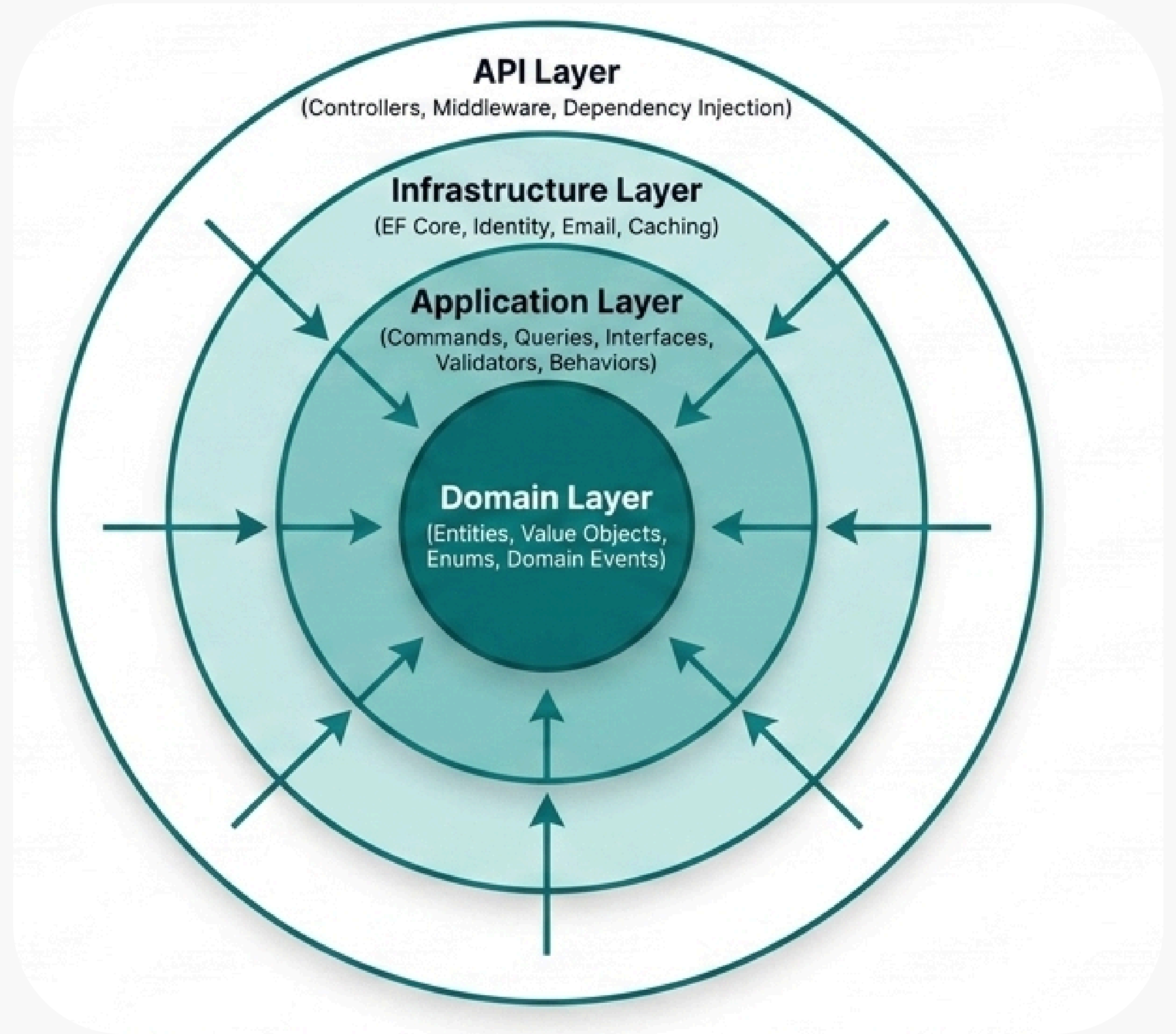
Advanced Features





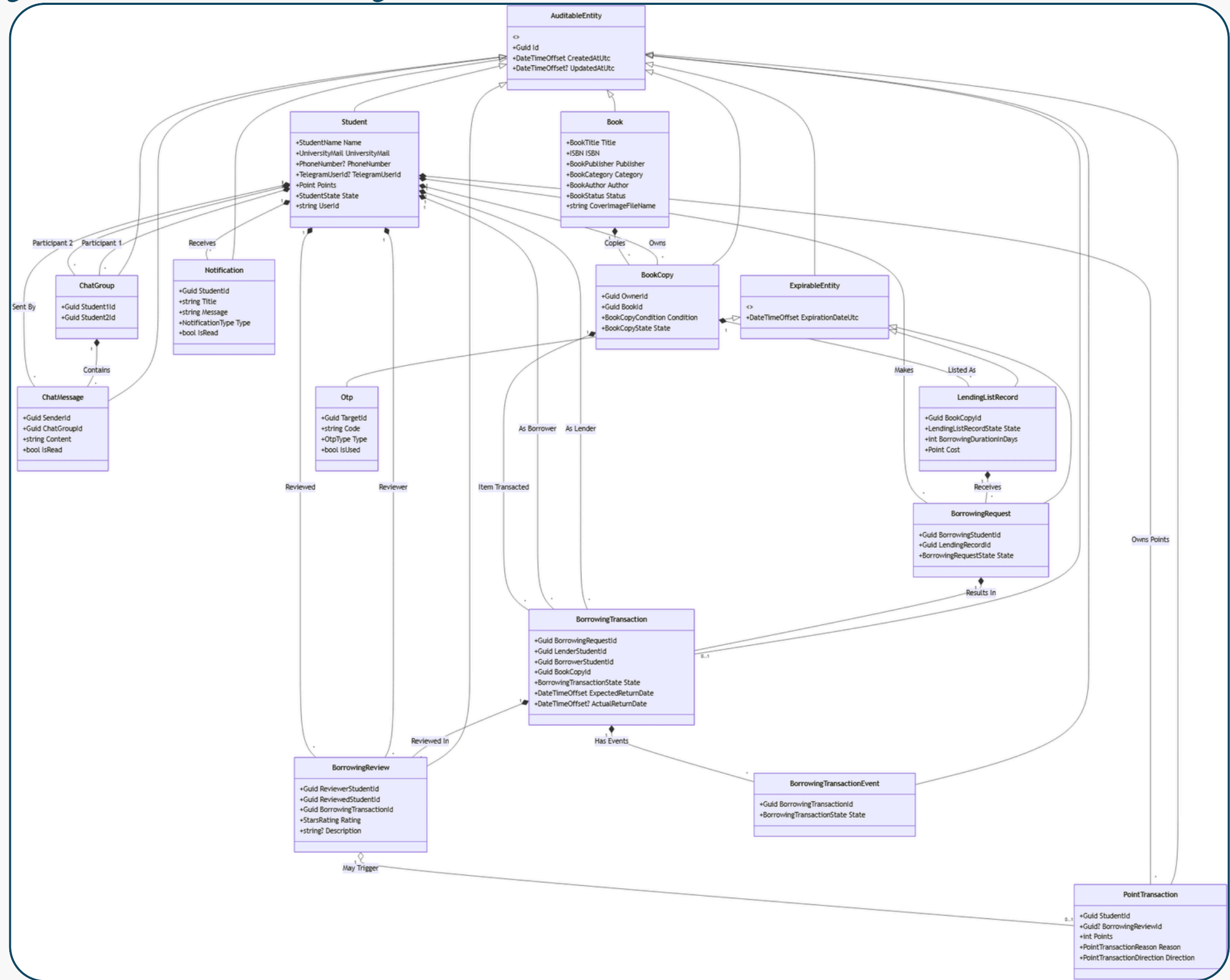
- **Api**
(Controllers, Middleware)
- **Infrastructure**
(EF Core, Identity, JWT)
- **Application**
(CQRS, MediatR, DTOs)
- **Domain**
(Entities, Value Objects, Events)

System Architecture — Clean Architecture



Domain Layer — Entity Hierarchy

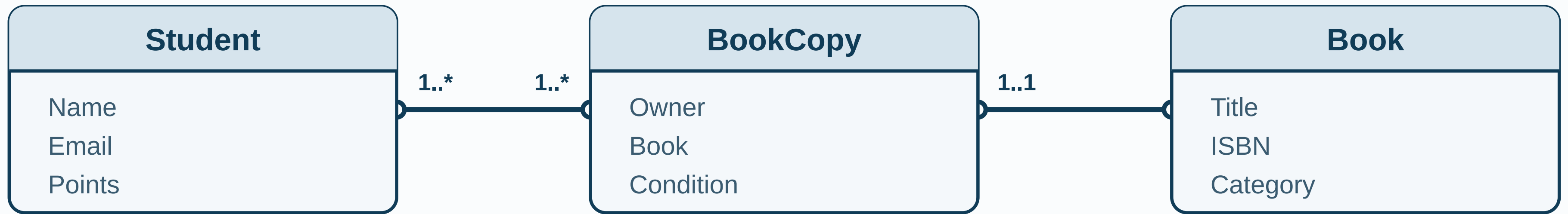
- **Strict base class hierarchy design.**
- **Supports Domain Events, automatic audit tracking, and time-bound behavior.**



Folder Structure

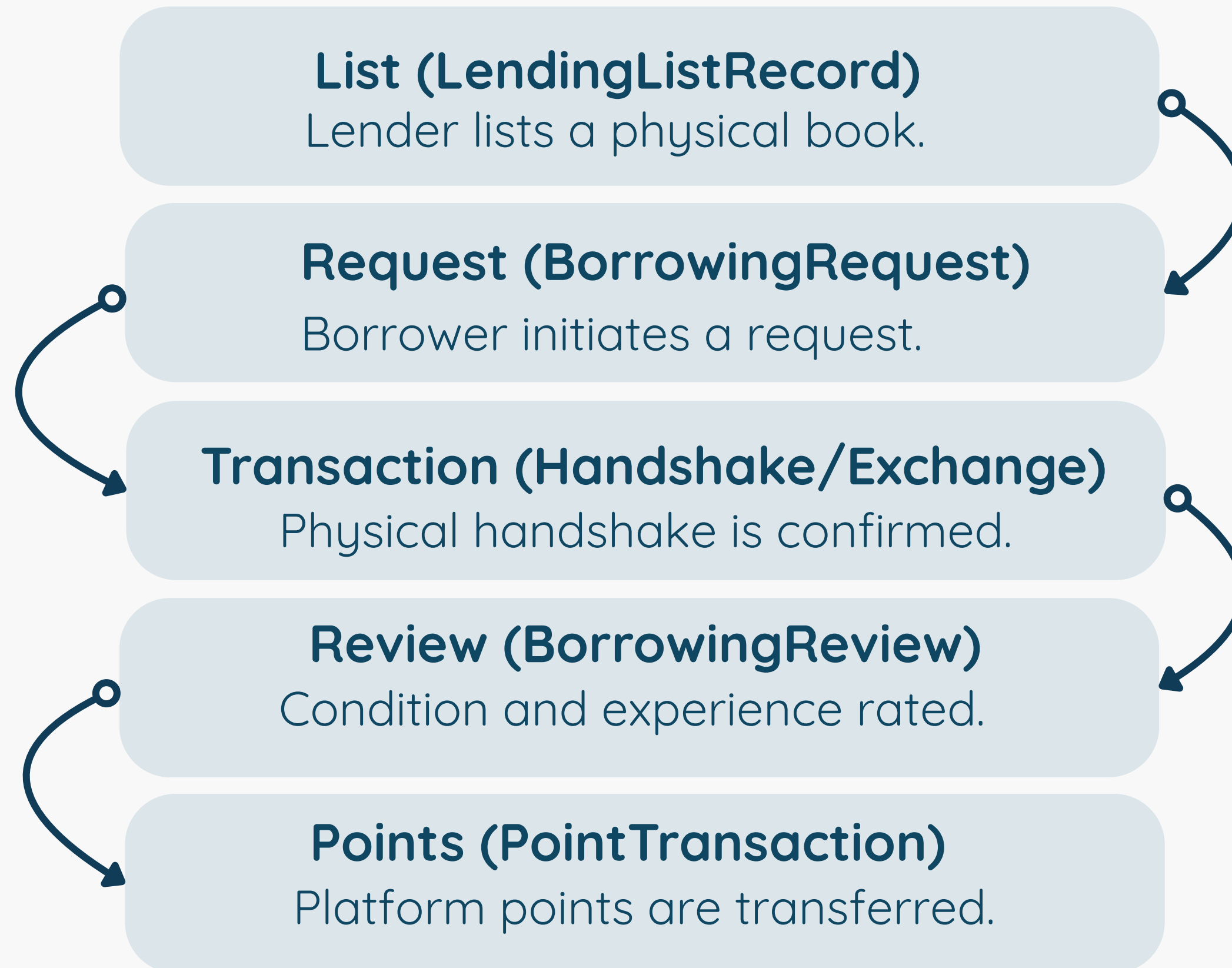
BookOrbit-backend/	
├── Code/	# Core Application Code
│ ├── BookOrbit.Api/	# Presentation Layer (Web API)
│ │ ├── Common/	# API-specific constants, mappers, etc.
│ │ ├── Contracts/	# API Request/Response Models (DTOs)
│ │ ├── Controllers/	# REST Controllers organized by entity
│ │ ├── Middlewares/	# Custom middleware (e.g., error handling)
│ │ ├── OpenApi/	# Swagger/OpenAPI configurations
│ │ ├── Services/	# Web-specific services
│ │ └── wwwroot/	# Static files (uploads, etc.)
│ └── BookOrbit.Application/	# Application Layer (Use Cases & Business Logic)
│ ├── Common/	
│ │ ├── Behaviours/	# MediatR Pipeline Behaviors (Validation, Logging)
│ │ ├── Exceptions/	# Application-level exceptions
│ │ ├── Interfaces/	# Abstractions for Infrastructure services
│ │ └── Models/	# Generic application models
│ └── Features/	# CQRS Features organized by Domain Entity
│ ├── Books/	
│ │ ├── Commands/	# Create/Update/Delete operations
│ │ ├── Queries/	# Read operations
│ │ └── Dtos/	# Data Transfer Objects
│ ├── BorrowingRequests/	
│ ├── BorrowingTransactions/	
│ ├── Chat/	
│ ├── Students/	
│ └── [Other Features...]	
├── BookOrbit.Domain/	# Domain Layer (Core Business Rules)
│ ├── Books/	# Aggregates & Entities
│ │ ├── Enums/	# Domain Enums
│ │ └── ValueObjects/	# Value Objects specific to Books
│ ├── BorrowingTransactions/	
│ │ ├── BorrowingReviews/	# Child entities
│ │ └── DomainEvents/	# Events emitted by the aggregate
│ ├── Common/	
│ │ ├── Entities/	# Base entities (e.g., BaseEntity)
│ │ └── Results/	# Result Pattern implementations
│ ├── Students/	
│ └── [Other Domain Entities...]	
├── BookOrbit.Infrastructure/	# Infrastructure Layer (External Concerns)
│ ├── BackgroundJobs/	# Hosted Services / Background Workers
│ ├── Common/	# Email templates, shared errors
│ ├── Data/	# Entity Framework Core specifics
│ │ ├── Configurations/	# FluentAPI configurations
│ │ ├── Interceptors/	# SaveChanges interceptors (e.g., Domain Events dispatching)
│ │ └── Migrations/	# EF Core Migrations
│ ├── Identity/	# Authentication & Authorization
│ │ └── Policies/	# Security & Access Policies
│ ├── Services/	# Concrete implementations of Application interfaces
│ │ ├── ChatServices/	# SignalR/Chat implementation
│ │ ├── ImageServices/	# File storage handling
│ │ └── OtpServices/	# SMS/Email OTP integrations
│ └── Settings/	# Configuration class mappings (Options pattern)
├── Tests/	# Test Suite
│ ├── BookOrbit.Application.SubcutaneousTests/	# Integration/Subcutaneous Tests
│ ├── BookOrbit.Application.UnitTests/	# Application Layer Unit Tests
│ └── BookOrbit.Domain.UnitTests/	# Domain Layer Unit Tests
└── Docs/	# Project Documentation
└── Diagrams/	# System and Architecture Diagrams

Domain Entities — Student, Book & BookCopy

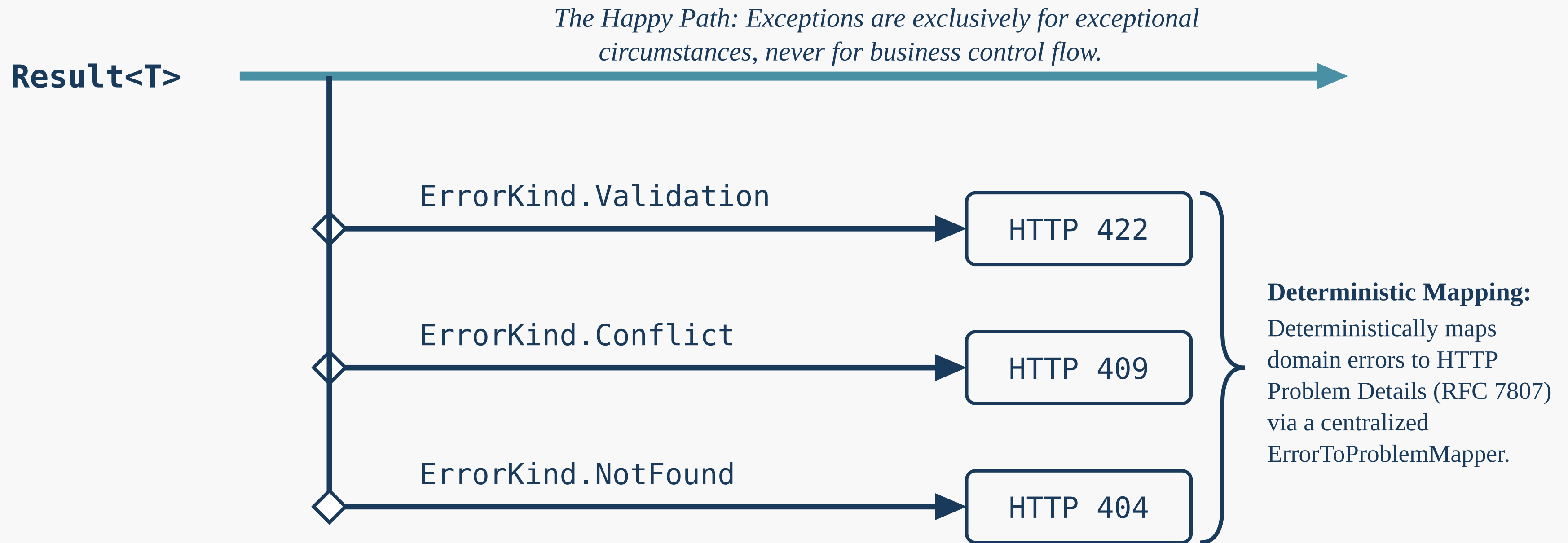


- **Student:** Points economy, verified identity.
- **Book:** Master catalog entity.
- **BookCopy:** Physical copy tracking condition.

The Lending & Borrowing Workflow



Exception-less Control Flow: Railway-Oriented Programming



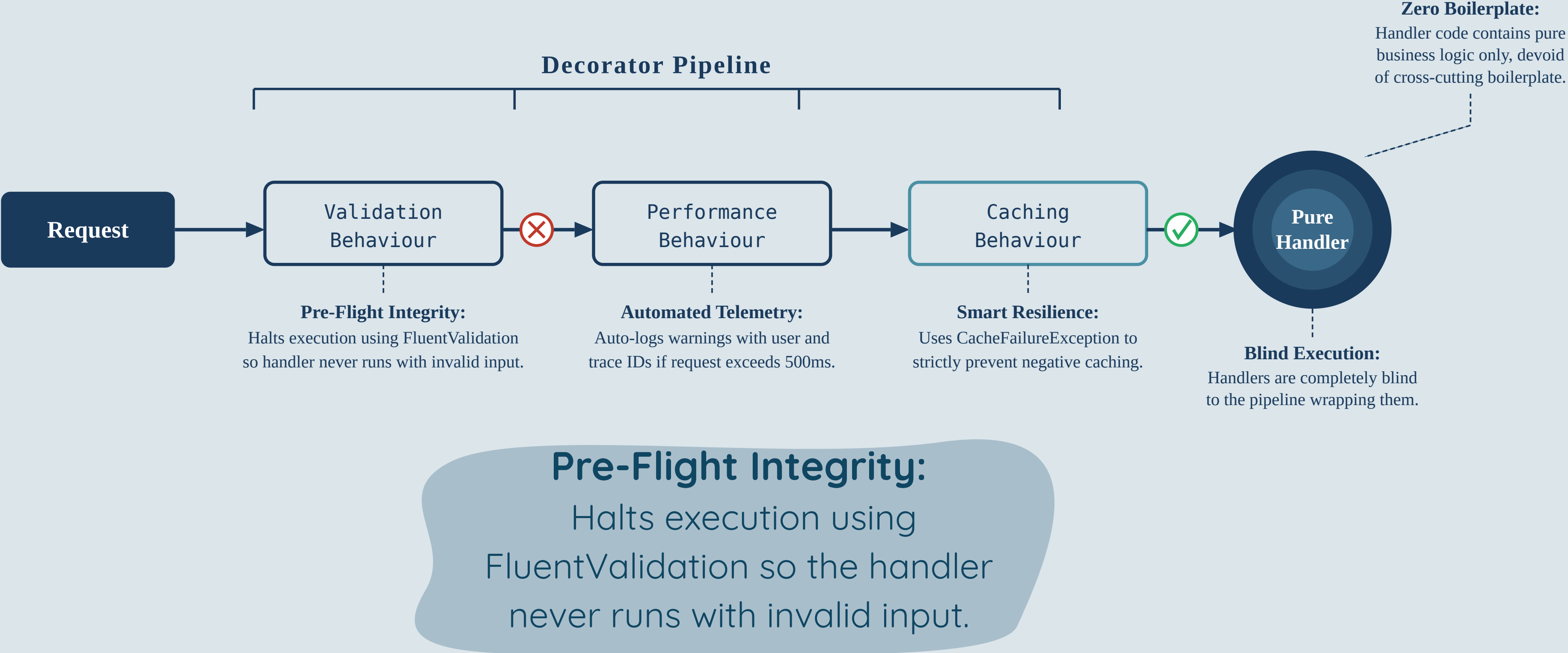
- **Forced Acknowledgment**

Forces the Presentation layer to explicitly acknowledge and handle every potential failure path.

- **No Hidden Modes**

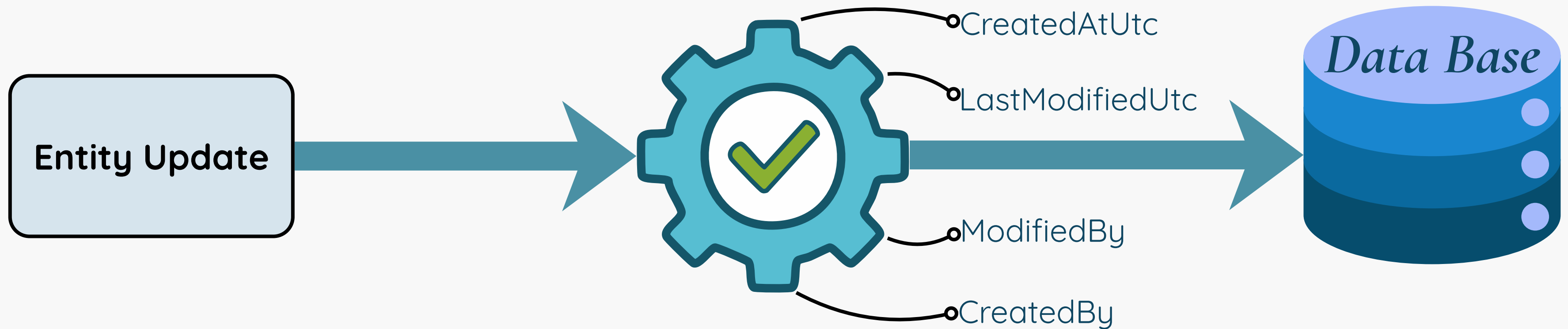
Delivers a 100% predictable API contract that eliminates unpredictable bubbling and hidden failure modes.

Application Pipeline: MediatR Behaviors



Smart Persistence: EF Core Interceptors

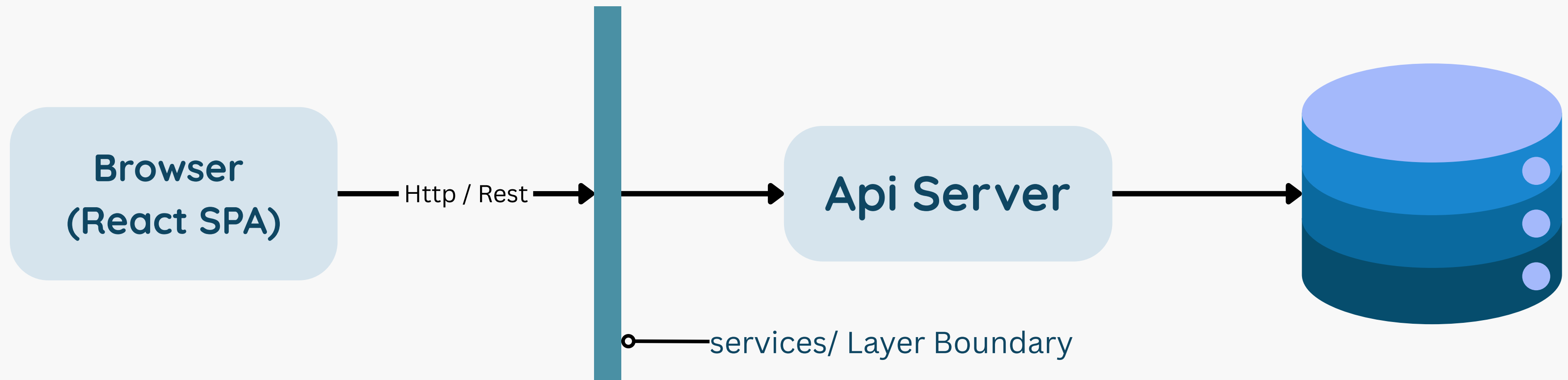
Global Hook: Hooks directly into `SaveChangesAsync` via a globally registered `AuditableEntityInterceptor`.



Automatic Injection: Automatically injects timestamps and user data on every single entity state change.

Transactional Consistency: Dispatches Domain Events via MediatR strictly before the database commit to guarantee transactional consistency.

Frontend Architecture & Tech Stack



React v19 & Tailwind CSS v3.4 deliver a highly responsive, utility-driven Single Page Application.

React Router v7 handles all client-side navigation and protected route enforcement.

Framer Motion & Lucide React provide micro-interactions and consistent iconography without bloat.

Feature-based architecture organizes the src/ directory by domains like pages/auth/ and pages/lending/.

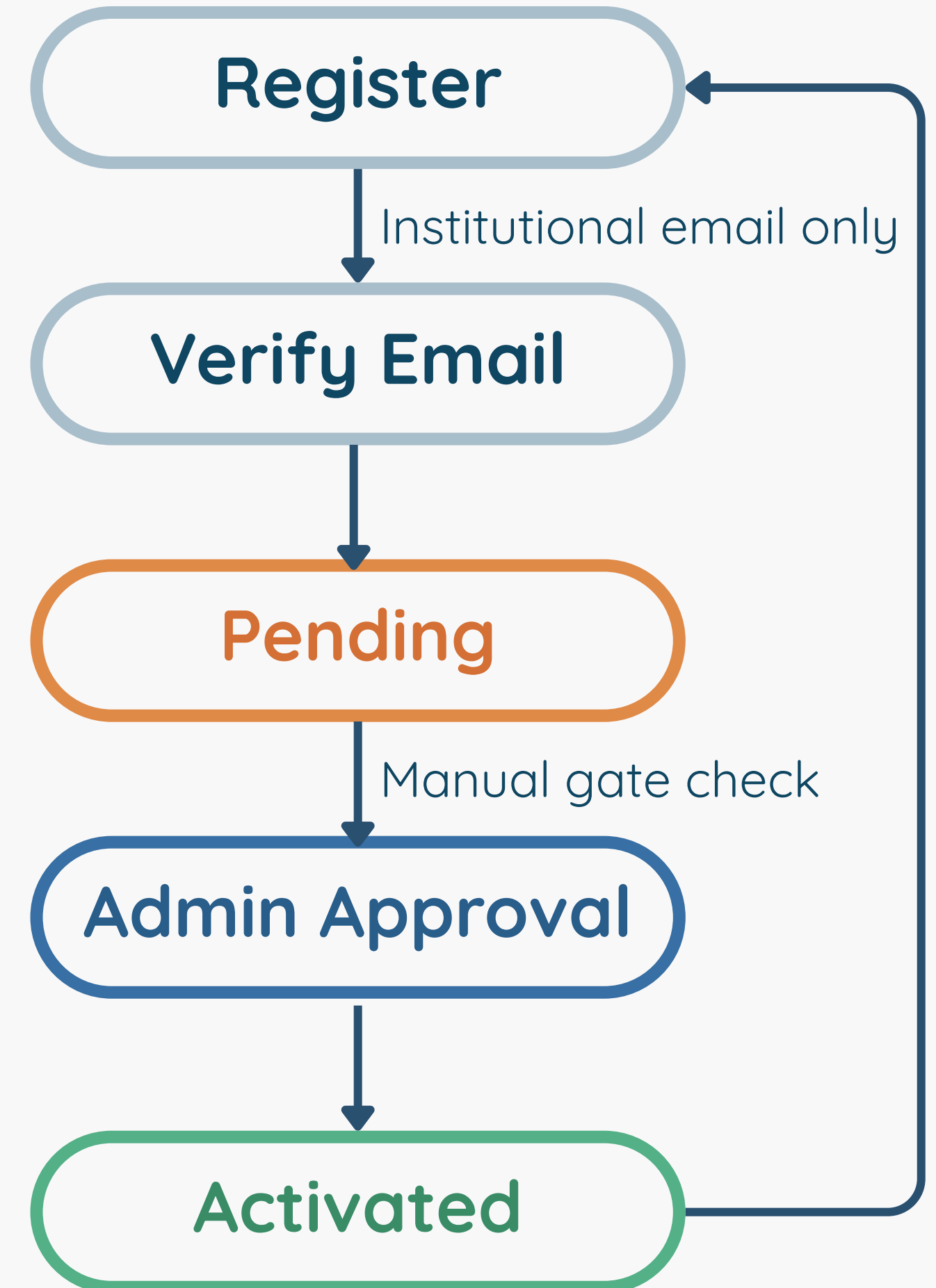
Service layer isolation ensures the UI never directly calls fetch, routing all API traffic through services/.

Testing Library enforces reliability across critical UI components and custom hooks.

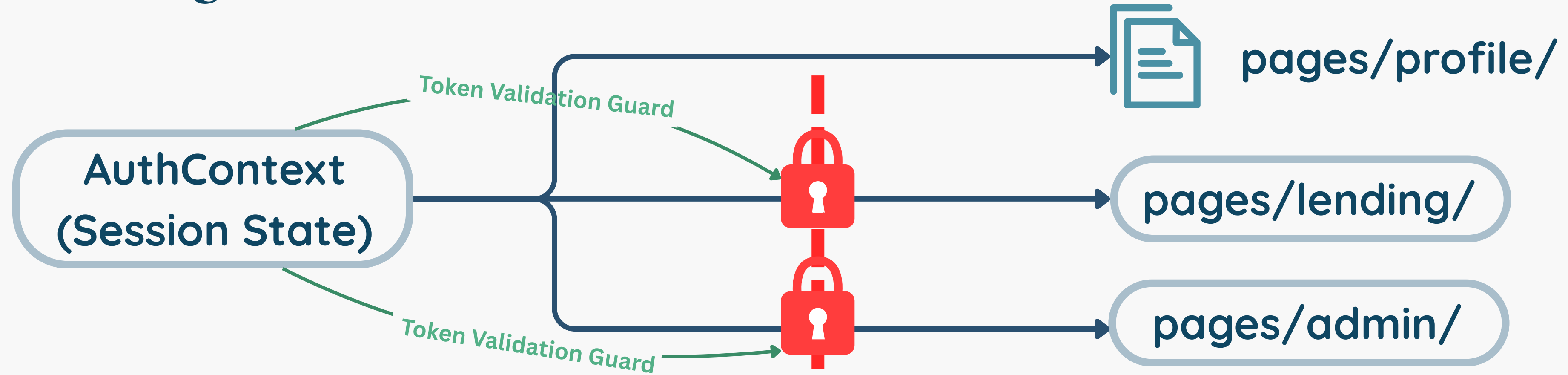
Authentication Flow & Session Management



- **University email gating**
strictly limits registration to verified institutional domain addresses.
- **JWT dual-token strategy**
isolates the short-lived access token from the secure, long-lived refresh token.
- **Silent refresh**
runs automatically via AuthContext, ensuring users are never interrupted mid-session.
- **Environment keys**
securely isolate token identifiers away from the compiled application bundle.
- **Manual gatekeeping**
requires Admin approval to finally transition an account to activated.



Routing & State Architecture



Client-side routing via React Router v7 prevents full-page reloads while managing complex navigational state.

Guard checkpoints ensure no protected view renders without first passing a strict authentication check.

Single source of truth is securely maintained by AuthContext for all session data across the application.

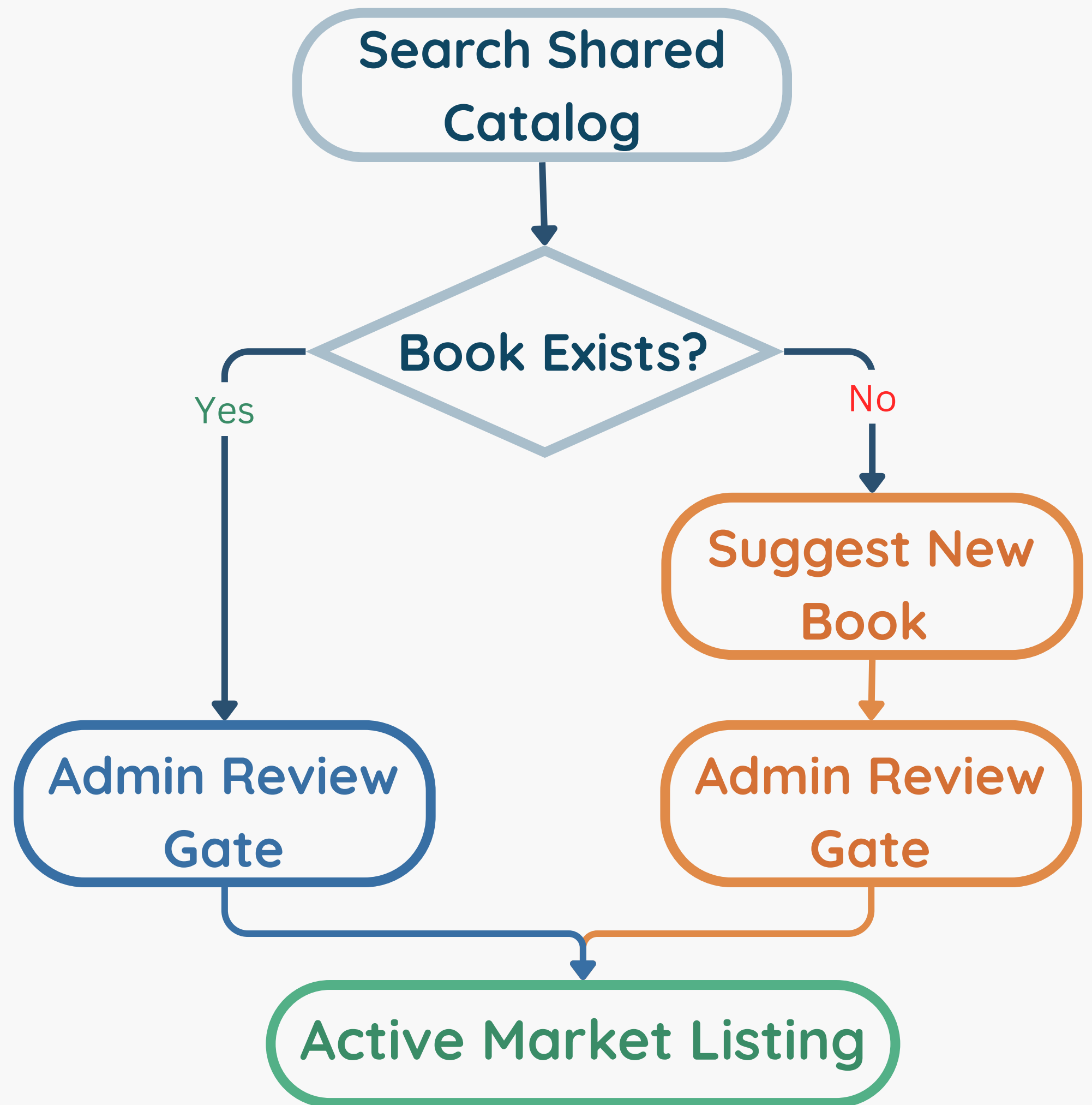
API boundary enforcement routes everything through services/, creating a single point of configuration.

Custom hooks extract complex business logic out of UI components into reusable, isolated functions.

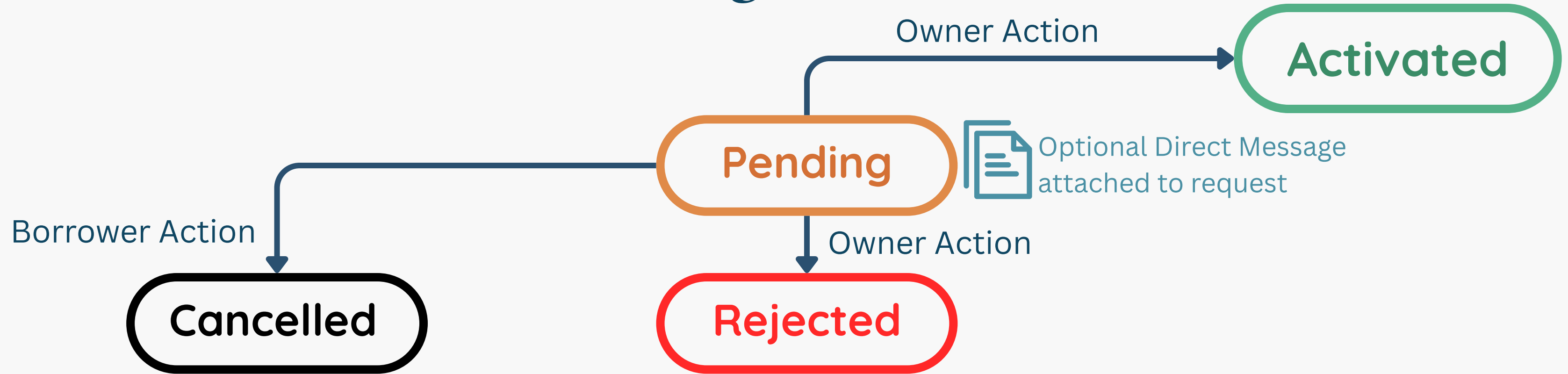
Centralized configuration makes adapting to backend URL changes a single-line adjustment.

Core User Flow: Listing a Book

- **Catalog-first architecture**
requires students to search the shared database before creating new entries.
- **Physical copy attachment**
securely links a user's specific physical book to the centralized master record.
- **New entry suggestions**
trigger when a book is missing, pushing the data to an admin review queue.
- **Asynchronous approval**
prevents unverified or duplicate books from cluttering the shared market.
- **Explicit listing action**
formally moves a verified physical copy into the active lending market.



Core User Flow: Borrowing a Book



Faceted browsing allows students to filter the lending market by title, category, and real-time availability.

State transition controls grant the owner exclusive rights to move a request from pending to accepted or rejected.

Asynchronous requests allow borrowers to send targeted pings with optional direct messages.

Proactive cancellation allows the borrower to abort the request unilaterally before an owner responds.

Instant notification alerts the physical owner that their actively listed copy has a prospective borrower.

Physical handover occurs off-platform only after the accepted state is mutually confirmed.

Admin Panel & Platform Control

Permission

Standard Student Role

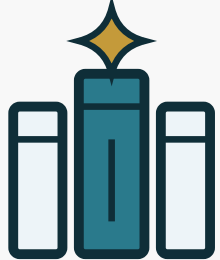
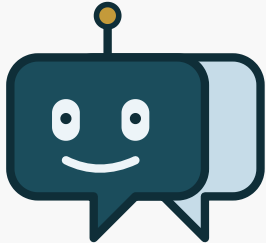
Administrator Role

- Browse Lending Market
- Access pages/admin/ routes
- Approve New Book Suggestions
- Ban / Unban User Accounts



- **Isolated administrative domain** securely houses all governance tools exclusively under the pages/admin/ directory.
- **Route-level authorization** strictly blocks standard students from accessing any administrative endpoints.
- **Granular account moderation** empowers admins to approve, reject, ban, or unban user profiles.
- **Route-level authorization** strictly blocks standard students from accessing any administrative endpoints.
- **Catalog hygiene enforcement** routes all new book suggestions through a manual admin approval gate.

Future Program Roadmap

- *RECOMMENDATION SYSTEM.* 
- *AI OPTIMIZES IMAGES OF BOOKS CAPTURED.* 
- *AI-POWERED SEARCH: USER DOESN'T HAVE TO SEARCH BY EXACT NAME.* 
- *INTEGRATE AN AI CHATBOT.* 

.....



Thank you!



.....

Any Question?